UNIVERSITY RESEARCH SYSTEM

Prepared by: Oleg Ukhov, Saken Ayan
Course: Object Oriented Programming Final Project
Date: May 2026
Institution: University

TABLE OF CONTENTS

1. Title Page and Team Information
2. Introduction
3. System Architecture and Design Overview
4. Class Descriptions and Hierarchy
5. Custom Exceptions
6. Design Patterns Explained
7. Demo Scenarios and Console Usage
8. Problems and Project Management
9. Technical Achievements and Limitations
10. Conclusion

SECTION 1: TITLE PAGE AND TEAM INFORMATION

University Research System
Final Project Report Part C Demo

Team Members
Oleg Ukhov - Project Lead, wrote all core classes and systems
Saken Ayan - Implemented data serialization and storage layer
Shyngyzkhan Nurlan (retake) - Was responsible for report and Main console class, did not complete

Project Date: May 2026
Programming Language: Java
Type: Console based application
Duration: Multiple weeks with coordination challenges

SECTION 2: INTRODUCTION

Purpose of the System
The University Research System is a comprehensive Java application designed to manage academic and research activities at a university. The system handles multiple user roles and implements complex business logic including course management, student registration with credit limits, grading systems, research paper publishing with h-index calculation, journal subscriptions using observer pattern, technical support ticketing, and news management.

Who Uses the System
The system serves multiple user types. Students use it to register for courses, view marks and transcripts, rate teachers, submit technical requests, and manage organization

memberships. Teachers use it to manage courses, input student marks, manage attendance, send complaints to managers, and engage in research activities. Managers approve course registrations, assign courses to teachers, create statistical reports, manage news items, and oversee the system. Administrators manage user accounts and view system logs. Tech support specialists handle technical requests from users. All of these roles interact with a shared database that persists to text files on disk.

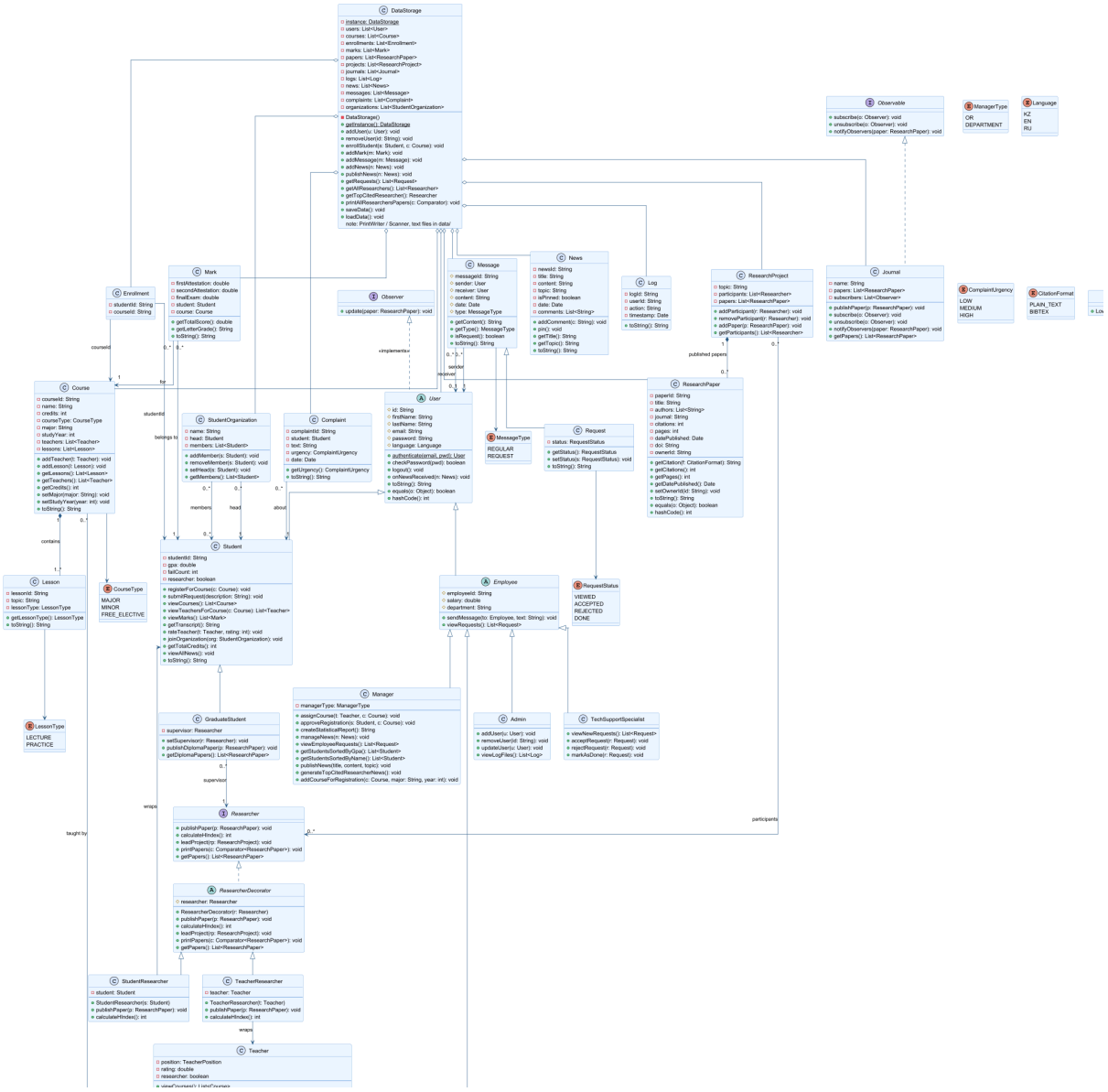
Key Features
Course registration with constraints on maximum credits per student and maximum failures allowed
Marking system with three components: first attestation, second attestation, final exam
Research paper management with citation counting and h-index calculation
Journal subscriptions with observer pattern for paper notifications
News system with automatic pinning of research topic news
Technical support request lifecycle management
Serialization of all data to text files for persistence
Multi language support for English, Russian, Kazakh
Authentication required for all users

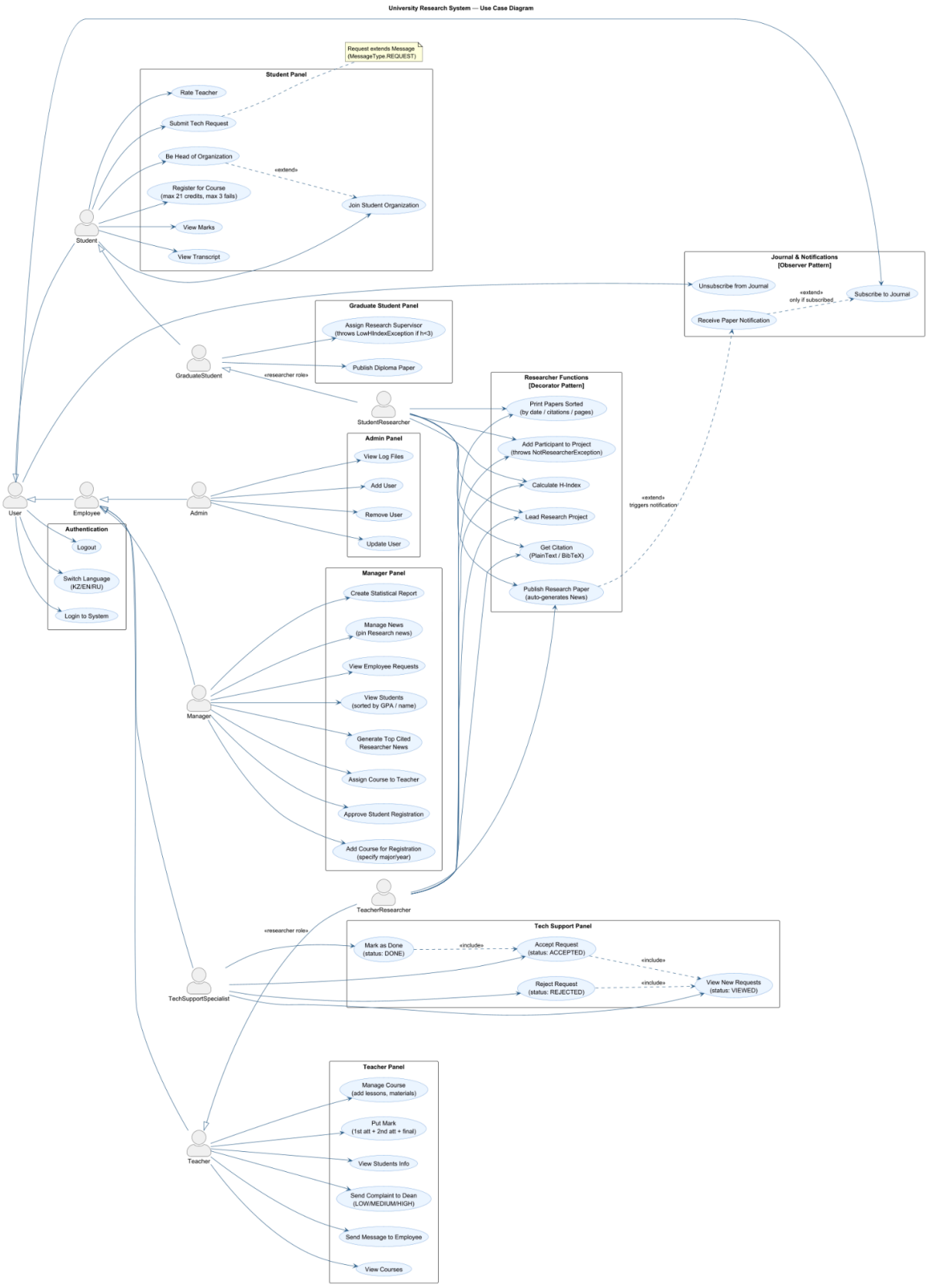
SECTION 3: SYSTEM ARCHITECTURE AND DESIGN OVERVIEW

Overall Architecture
The system follows a layered architecture with clear separation of concerns. At the bottom is the DataStorage singleton which manages all data persistence. The middle layer contains all domain classes organized by responsibility: user hierarchy, course and enrollment management, research entities, and messaging. The top layer is the console UI in the Main class which handles user interaction and delegates operations to domain classes.

UML Diagrams
Below are the final versions of the class diagram and use case diagram that reflect the implemented system.

University Research System — Class Diagram





The system uses object oriented principles throughout. Low coupling is achieved by using interfaces like Researcher and Observable. High cohesion is maintained by grouping related responsibilities in classes like ResearcherDecorator or DataStorage.

Core Design Decisions
Every design choice was made with specific reasoning. The most important decision was making Researcher a decorator instead of an interface or abstract class. This is explained in detail in the design patterns section. Another critical decision was using a singleton DataStorage to ensure only one data source exists. Alternative approaches like passing storage as parameters everywhere would create tight coupling throughout the system.

The exception design uses checked exceptions where the caller might reasonably recover, and unchecked exceptions where the condition is a logic error. Custom runtime exceptions like LowHIndexException and NotResearcherException signal constraint violations that should fail fast.

User Role Hierarchy
The system implements a clear user hierarchy using inheritance. At the top is the abstract User class that all users inherit from. User provides common functionality like authentication, language switching, and basic data access.

Employee extends User and adds employee specific fields like employeeId and salary. From Employee, three concrete types inherit: Teacher, Manager, and Admin. These three roles have different permissions and responsibilities.

Separately, Student extends User directly. GraduateStudent then extends Student to add supervisor and diploma paper functionality.

TechSupportSpecialist is a special role that extends Employee and handles the request lifecycle.

SECTION 4: CLASS DESCRIPTIONS AND HIERARCHY

The User Class Hierarchy

User Abstract Class
The User class is the root of the hierarchy. It contains common data: id, firstName, lastName, email, password, language. It provides authentication via a static method authenticate that takes email and password and returns the logged in user or null. The checkPassword method verifies a password. Language support is implemented via a translation method t that takes English, Kazakh, and Russian versions of text and returns the appropriate one based on the user's language setting.

The User class implements Observer interface so all users can receive notifications when research papers are published to journals they are subscribed to.

Employee Abstract Class
Employee extends User and adds fields for employeeId, salary, and department. It provides methods for sending messages to other employees and viewing incoming requests. Employees can submit technical support requests via submitRequest method.

Teacher Class
Teacher extends Employee and adds TeacherPosition enum field that can be TUTOR, LECTOR, SENIOR_LECTOR, or PROFESSOR. Teachers have a rating field that accumulates from student reviews. They can be marked as researchers using the researcher boolean field.

Teachers have methods to view their assigned courses, manage course content, put marks for students, view student information, send complaints to managers with urgency levels, and rate themselves based on student feedback.

Manager Class
Manager extends Employee and has a ManagerType field that is either OR or DEPARTMENT, representing different management scopes. Managers assign courses to teachers, approve student course registrations, add new courses to the system specifying which major and year they are for, create statistical reports on academic performance, manage news items, publish automatic news about top cited researchers, and view employee requests.

Admin Class
Admin extends Employee and provides user management capabilities. Admins can add new users, remove users, update existing user information, and view system logs that track all user actions.

TechSupportSpecialist Class
TechSupportSpecialist extends Employee and handles technical request workflows. When a request is submitted, it gets status VIEWED. Tech support can then accept it setting status ACCEPTED, reject it setting status REJECTED, or complete it setting status DONE.

Student Class
Student extends User directly. It has fields for studentId, gpa, failCount to track how many times they failed courses, a researcher flag, studyYear, faculty, and a graduated flag.

Students can register for courses, view their courses, view teacher information for a course, view their marks, get transcripts, rate teachers, join student organizations, mark attendance, and view their attendance list.

Student provides getTotalCredits method that sums up credits from all enrolled courses to enforce the maximum 21 credit limit. When a student fails a course for the third time, further registration is blocked.

GraduateStudent Class
GraduateStudent extends Student and adds a supervisor field that must be a Researcher, and a list of diploma paper IDs. The setSupervisor method enforces that the supervisor has h-index of at least 3, throwing LowHIndexException if not. GraduateStudent can publishDiplomaPaper to add research papers as their diploma thesis.

Research Related Classes

Researcher Interface
Researcher is an interface that defines the research behavior a user can have. It specifies methods: publishPaper to add a paper to the researcher's list, calculateHIndex to compute h-index, leadProject to create or lead a research project, printPapers to display papers sorted by a comparator, and getPapers to get the list of papers.

Researcher is not a class itself but a role that users can take on via the decorator pattern. Any User can become a Researcher by wrapping themselves in TeacherResearcher or StudentResearcher decorator.

ResearcherDecorator Abstract Class
ResearcherDecorator is an abstract decorator that implements Researcher interface. It holds a reference to the decorated Researcher (which is another decorator or the base researcher object), an ownerId, and a list of projects.

The decorator adds research behavior by delegating to DataStorage. When publishPaper is called, it adds the paper to DataStorage, publishes a news item about it, notifies journal subscribers, and pins the news if topic is Research.

TeacherResearcher Class
TeacherResearcher is a concrete decorator that wraps a Teacher. It adds research capability to the teacher without modifying the Teacher class itself. This allows a teacher with position PROFESSOR to be a researcher simultaneously while still being a teacher.

StudentResearcher Class
StudentResearcher is a concrete decorator that wraps a Student. It allows students, especially graduate students, to conduct research while remaining students. This is useful for Master and PhD students who are both students and active researchers.

ResearchPaper Class
ResearchPaper represents a published academic paper. It has paperId, title, authors list, journal name, citations count, pages count, datePublished, doi identifier, and ownerId to track which researcher owns it.

The getCitation method returns a formatted citation string in either PLAIN_TEXT format showing authors and journal, or BIBTEX format suitable for use in academic documents.

The compareTo method implements Comparable interface to enable natural ordering by date published. This allows sorting papers chronologically.

ResearchProject Class
ResearchProject has projectId, topic, a list of participants who must all be Researchers, and a list of papers published under the project. When adding a participant, it checks if the user is a Researcher and throws NotResearcherException if not.

Academic Classes

Course Class

Course represents an academic course with courseId, name, credits count, courseType from enum CourseType that is MAJOR, MINOR, or FREE_ELECTIVE, major field for which major the course belongs to, studyYear that specifies which year of study the course is for, and lists of teachers and lessons.

Courses can have multiple teachers for lecture and practice sessions separately. Courses contain one or more lessons that meet at specific times.

Lesson Class
Lesson represents a single meeting of a course. It has lessonId, topic, lessonType as LECTURE or PRACTICE, dayOfWeek, startTime, and endTime. This allows the system to track when lessons occur for scheduling and attendance purposes.

Mark Class
Mark represents a student's grade in a course. It stores the student reference, course reference, and three score components: firstAttestation, secondAttestation, and finalExam, each a double from 0 to 100.

getTotalScore sums the three components. getLetterGrade converts the total to A, B, C, D, F letter grade based on standard university grading scale. Marks are the main way to track academic progress.

Other Important Classes

Enrollment Class
Enrollment is a simple data class that represents a student enrolled in a specific course and lesson. It stores studentId, courseId, and lessonId. The system checks for duplicate enrollments to prevent a student from being enrolled twice.

Attendance Class
Attendance records track whether a student attended a lesson. Each attendance record links a student to a course and lesson and has a boolean present flag and an attendance date.

The system can open attendance sessions for a course and lesson, allowing students to mark themselves present. This is tracked separately from marks.

Complaint Class
Complaint represents a formal complaint from a teacher about a student to the manager or dean. It has complaintId, the student object, complaint text, urgency level as an enum of LOW, MEDIUM, HIGH, and the date filed. Complaints are logged in DataStorage.

StudentOrganization Class
StudentOrganization represents student clubs or groups. It has organizationId, name, a head Student who runs it, and a list of member Students. Students can join organizations and be promoted to head of organization.

News Class

News represents a system announcement or news item. It has newsId, title, content, topic, isPinned flag, date published, and a list of comments.

News with topic "Research" is automatically pinned to appear at the top of the news feed. Other news items appear below in chronological order. Users can add comments to news items to discuss them.

Message Class
Message represents a communication between users. It stores messageId, sender, receiver, content, date, and type from enum MessageType which is either REGULAR for normal messages or REQUEST for technical support requests. Messages are the base class for Request.

Request Class
Request extends Message and adds a status field from enum RequestStatus that can be VIEWED when first seen, ACCEPTED when tech support takes it, REJECTED when declined, or DONE when completed.

Log Class
Log records user actions in the system. It stores logId, userId of the user who performed the action, action description, and timestamp. Admin can view all logs to track system usage.

Journal Class and Observer Pattern
Journal implements Observable and represents an academic journal that publishes research papers. It has journalId, name, list of papers, and list of subscribers who are Users.

When a paper is published to the journal via publishPaper method, the journal notifies all subscribers via notifyObservers method. Each subscriber's update method is called to notify them of the new paper.

DataStorage Singleton Class
DataStorage is implemented as a singleton to ensure exactly one instance exists globally. It uses a static instance variable and private constructor. getInstance returns the singleton instance.

DataStorage maintains lists of all domain objects: users, courses, enrollments, marks, papers, projects, journals, logs, news, messages, complaints, organizations, attendances, and more.

It provides methods to add, update, remove, and query these objects. When objects are added or modified, DataStorage manages the complete lifecycle and also handles saving to files.

DataStorage has special methods like getAllResearchers that returns all users who are decorated with researcher behavior, getTopCitedResearcher that finds the most cited researcher among all users, and printAllResearchersPapers that displays all research papers sorted by a provided comparator.

The saveData method writes all objects to text files in a data directory using PrintWriter, encoding special characters so the text format is safe. The loadData method reads the files back using Scanner and reconstructs all objects.

SECTION 5: CUSTOM EXCEPTIONS

Why Custom Exceptions Matter
Custom exceptions allow the system to signal specific error conditions that the caller can handle appropriately. They document what can go wrong at each method. They also distinguish logic errors from unexpected runtime errors.

The system uses four custom exceptions, all extending RuntimeException to be unchecked. This signals that they represent violated constraints that should fail fast and be fixed in the code, not handled at runtime.

LowHIndexException
When thrown: thrown in GraduateStudent setSupervisor method if the supervisor Researcher has h-index less than 3
Why needed: the system requirement states that only experienced researchers with sufficient publication impact can supervise graduate students. An h-index of at least 3 means the researcher has at least 3 papers with at least 3 citations each, showing minimal impact and credibility.

Code example:
```
  public void setSupervisor(Researcher r) throws LowHIndexException {
    if (r.calculateHIndex() < 3)
      throw new LowHIndexException("Supervisor h index too low. Required at least 3");
    this.supervisor = r;
  }
```

CourseOverloadException
When thrown: thrown in Student registerForCourse method if adding another course would exceed 21 total credits
Why needed: university policy limits student course load to 21 credits per semester to ensure students do not overload themselves and can maintain quality work. This is enforced as a hard constraint.

Code example:
```
  public void registerForCourse(Course c, Lesson l) throws CourseOverloadException {
    if (getTotalCredits() + c.getCredits() > 21) {
      throw new CourseOverloadException("Cannot exceed 21 credits. Current " +
getTotalCredits());
    }
    // proceed with registration
  }
```

CourseFailLimitException

When thrown: thrown in Student registerForCourse if the student has failed the course more than 3 times
Why needed: the system requirement specifies students cannot fail the same course more than 3 times. After 3 failures, the student must drop the course. This enforces academic standards.

Code example:

```
public void registerForCourse(Course c, Lesson l) throws CourseFailLimitException {
  int fails = countFailsForCourse(c.getCourseId());
  if (fails >= 3) {
    throw new CourseFailLimitException("You failed this course 3 times already");
  }
  // proceed with registration
}
```

NotResearcherException
When thrown: thrown in ResearchProject addParticipant method if the user is not decorated with Researcher behavior
Why needed: only active researchers can join research projects. This prevents regular students or staff from participating in research as primary investigators. It enforces the role separation.

Code example:

```
public void addParticipant(Researcher r) throws NotResearcherException {
  if (r == null || !(r instanceof ResearcherDecorator)) {
    throw new NotResearcherException("Only researchers can join projects");
  }
  participants.add(r);
}
```

SECTION 6: DESIGN PATTERNS EXPLAINED

Design patterns are reusable solutions to common design problems. This system implements four patterns: Singleton, Decorator, Observer, and Strategy. Each was chosen deliberately to solve a specific architectural challenge.

Pattern 1: Singleton Pattern - DataStorage

What It Does
Singleton pattern ensures exactly one instance of a class exists during program execution. It provides a global access point via getInstance static method.

Why DataStorage Needed It
The system has one database of all users, courses, marks, papers, and other entities. Having multiple DataStorage instances would mean different parts of the program see inconsistent data. For example, if one part saved a new user but another part did not know about it, operations would fail.

Alternative Approaches Considered
Passing DataStorage as parameter through every method call would work but creates coupling. Every method signature becomes polluted with storage parameter. Passing storage through a context object is cleaner but still requires passing everywhere.

Global variable could work but is considered poor practice and hard to test. Global mutable state makes testing difficult since tests cannot isolate independently.

Implementation Details
DataStorage has a private static instance variable initialized to null. The private constructor prevents direct instantiation from outside. The getInstance method checks if instance is null and creates it on first call, then returns the singleton instance.

```
public class DataStorage {
  private static DataStorage instance = null;

  private DataStorage() {
    // prevent instantiation
  }

  public static DataStorage getInstance() {
    if (instance == null) {
      instance = new DataStorage();
    }
    return instance;
  }
}
```

Why This Pattern is Superior
All code accesses the same DataStorage instance. Adding a user in one method means all other methods see that user immediately. The single point of access makes it easy to add logging, caching, or other cross cutting concerns later.

Pattern 2: Decorator Pattern - Researcher Role

What It Does
Decorator pattern wraps an object and adds new functionality without modifying the original class. A decorator implements the same interface as the object it wraps, so it can be used anywhere the original is used.

Why Researcher Needed It
The system requirement stated that Teachers can be researchers, Students can be researchers, and employees who are neither can be researchers. But Java does not support multiple inheritance. A Teacher cannot inherit from both Employee and Researcher.

A Teacher who teaches courses but also publishes papers needs both Teacher and Researcher behavior. Using decorator, a Teacher object is wrapped in TeacherResearcher which adds research methods. The TeacherResearcher passes through teacher methods and adds research methods.

Alternative Approaches Considered and Why They Failed
Interface approach: Make Researcher just an interface. Each class implements it independently. Problem: code duplication. The h-index calculation logic would be copied into Teacher, Student, and other classes. When the algorithm changes, every class needs updating.

Abstract class approach: Create abstract class Researcher that all user types inherit from. Problem: Java single inheritance does not allow this without modifying the user hierarchy. Would need to move research into every branch.

Mixin approach: Use composition without the decorator structure. Just add a researcher field to each class. Problem: adds noise to every class signature. Also does not cleanly separate concerns.

Implementation Details
ResearcherDecorator implements Researcher interface and holds reference to decorated Researcher. It implements all Researcher methods by delegating to DataStorage or to the wrapped researcher.

```
public abstract class ResearcherDecorator implements Researcher {
  protected Researcher researcher;
  protected String ownerId;
  protected List<ResearchProject> projects;

  public void publishPaper(ResearchPaper p) {
    DataStorage ds = DataStorage.getInstance();
    p.setOwnerId(ownerId);
    ds.addPaper(p);
    News n = new News("Research", "New paper published");
    ds.publishNews(n);
  }

  public int calculateHIndex() {
    // implementation details below
  }
}
```

TeacherResearcher wraps a Teacher and StudentResearcher wraps a Student. Both provide researcher functionality while preserving original user type.

Why This Pattern is Superior

Separation of concerns: research logic is separate from user types. A Teacher is still a Teacher with all teaching methods. Research is added as a separate layer.

Flexibility: a user can become a researcher at any time by wrapping them in a decorator. They can be unwrapped later if they stop researching.

Reuse: the same ResearcherDecorator logic works for any user type that can research. No duplication of h-index or paper publishing code.

Pattern 3: Observer Pattern - Journal Subscriptions

What It Does
Observer pattern decouples event producers from listeners. When event occurs, producer notifies all registered observers without knowing who they are or what they do with the notification.

Why Journal Needed It
When a researcher publishes a paper, many users may want to know about it. Manager wants to see it for reports. Other researchers want to see it to build on the work. Students want to stay informed.

If publishers had to know about all interested parties and call methods on each one, it would create tight coupling. Publishers would need to know Manager, Student, Researcher classes. Adding new interest types would require modifying publisher code.

Alternative Approaches Considered
Polling: Each user periodically checks if new papers exist. Problem: inefficient and delayed notification. Users get updates only when they poll. Creates database load from constant checking.

Direct callback: Publisher keeps list of callbacks and calls them directly. Problem: not much different from observer, just less clean abstraction.

Event queue: Put events in a queue and users process them. Problem: adds complexity. Also does not solve coupling issue if queue must know all event types.

Implementation Details
Journal implements Observable interface with methods subscribe, unsubscribe, and notifyObservers. User implements Observer interface with update method.

When a paper is published to journal, journal calls notifyObservers which iterates subscribers list and calls update on each one, passing the new paper.

```
public class Journal implements Observable {
  private List<Observer> subscribers = new ArrayList<>();

  public void subscribe(Observer o) {
```

```
    subscribers.add(o);
  }

  public void notifyObservers(ResearchPaper p) {
    for (Observer o : subscribers) {
      o.update(p);
    }
  }
}

public abstract class User implements Observer {
  public void update(ResearchPaper p) {
    Message m = new Message();
    m.setContent("New paper: " + p.getTitle());
    // message delivered to user
  }
}
```

Why This Pattern is Superior
Loose coupling: publishers and subscribers do not know each other. Journal does not import User classes.

Dynamic subscriptions: users can subscribe and unsubscribe at runtime. No need to rebuild the system.

Multiple subscribers: one event can notify many users. All subscribers handle it independently.

Pattern 4: Strategy Pattern - Sorting Comparators

What It Does
Strategy pattern defines a family of algorithms, makes each one encapsulated, and makes them interchangeable. The calling code selects which strategy to use at runtime.

Why Sorting Needed It
Researcher printPapers method needs to display papers in different orders: by date, by citations, by page count. The system also prints all researchers papers sorted by chosen strategy.

If sorting was hardcoded as method, system would need printPapersByDate, printPapersByCitation, printPapersByLength methods. Adding new sort order requires new method. Calling code must know all available sorts.

Alternative Approaches Considered
Multiple methods: printByDate, printByCitation, etc. Problem: method explosion. System becomes hard to extend.

Sorting parameter string: "date", "citation", "length" checked in if-else. Problem: not type safe. Misspelling breaks code. Adding sort type requires modifying method.

Comparator objects: pass in Comparator interface. Problem: already using this approach, it is the Strategy pattern.

Implementation Details
Strategy is represented by Java Comparator interface. DateComparator, CitationComparator, and LengthComparator each implement Comparator with compare method.

printPapers method takes Comparator as parameter. It sorts papers using passed comparator without knowing which one.

```
public void printPapers(Comparator<ResearchPaper> c) {
  List<ResearchPaper> sorted = new ArrayList<>(papers);
  Collections.sort(sorted, c);
  for (ResearchPaper p : sorted) {
    System.out.println(p.getTitle() + " " + p.getCitations());
  }
}

class CitationComparator implements Comparator<ResearchPaper> {
  public int compare(ResearchPaper a, ResearchPaper b) {
    return Integer.compare(b.getCitations(), a.getCitations());
  }
}
```

Caller chooses strategy:
```
researcher.printPapers(new CitationComparator());
researcher.printPapers(new DateComparator());
```

Why This Pattern is Superior
Open Closed Principle: adding new sort strategy does not require modifying existing code. Just create new Comparator class.

Runtime selection: which strategy to use is decided at runtime based on user input or context.

Testing: each comparator is independent and testable in isolation.

SECTION 7: DEMO SCENARIOS AND CONSOLE USAGE

This section walks through 16 demo scenarios showing how to use the system. Each scenario explains what the user does, what the system outputs, why the feature matters, and includes placeholder for screenshot.

How to Build and Run
To compile the system, run this command in the project root directory:
  javac -d out -cp src src\Main.java

This compiles all source files from src directory and puts class files in out directory. The Main.java serves as entry point.

To run the system, execute:
  java -cp out Main

The system will display a login prompt. Enter credentials to authenticate as one of the sample accounts.

Sample Accounts Available
The system has predefined test accounts for demonstration:
  Student account: email student@uni password student
  Teacher account: email teacher@uni password teacher
  Manager account: email manager@uni password manager
  Admin account: email admin@uni password admin

DEMO SCENARIO 1: User Login and Authentication

Purpose
Demonstrate user authentication system and verify credentials are checked correctly.

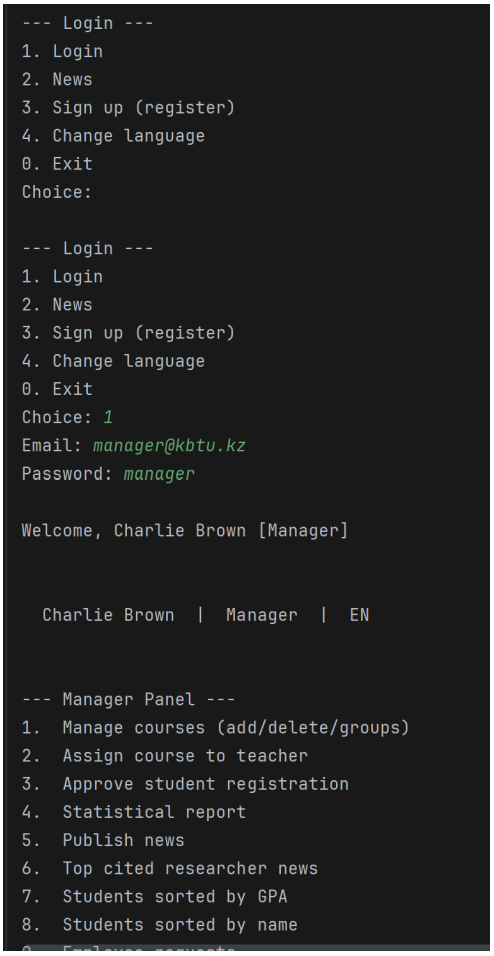
Steps
1. Run the system with command shown above
2. System displays login prompt
3. Enter email: student@uni
4. Enter password: student
5. System authenticates and displays welcome message

Expected Output
Login prompt appears asking for email and password. After entering credentials, system responds with:
  Welcome student First Last
  Login successful
  Today is Monday May 20 2026

Console shows main menu with student options like View Courses, Register for Course, View Marks, View Transcript, Rate Teacher, Join Organization, Submit Tech Request.

Why This Matters

Authentication is the foundation of access control. Every user must login before accessing system. System verifies email exists in database and password matches stored password. Without authentication, anyone could access another user's marks or submit complaints pretending to be someone else.



DEMO SCENARIO 2: Switch Language

Purpose
Demonstrate the multilingual support system that allows users to switch between English, Russian, and Kazakh.

Steps
1. After logging in as student, look for language option in main menu
2. Select option to change language
3. Choose language: EN, RU, or KZ
4. System confirms language change
5. Return to main menu to see labels in new language

Expected Output
System displays prompt like:
Select language:
1. English (EN)

2. Russian (RU)
3. Kazakh (KZ)
Enter choice: 1

System responds:
Language set to English

All subsequent menu items appear in English. If you select Russian, menu items like "View Courses" become "Просмотр Курсов" and labels change throughout the session.

Why This Matters
University serves students and staff from different backgrounds. Kazakh is primary language of Kazakhstan. Russian is widely spoken. English is important for international students and academic papers. Language switching improves accessibility and user experience. Implementation uses helper method t(english, russian, kazakh) that returns appropriate version based on current language setting.

```
--- All users ---
90. Switch language KZ/EN/RU
91. Subscribe to journal
92. Unsubscribe from journal
93. View news
94. Publish paper to journal (notify)
95. Logout
96. View my messages
97. Change password
0.  Save & Exit
Choice: 90
1.KZ  2.EN  3.RU
1
Тіл өзгертілді: KZ
Language: KZ


  Charlie Brown  |  Менеджер  |  KZ


--- Менеджер панелі ---
1.  Курс басқару
2.  Курс тағайындау
3.  Тіркеуді бекіту
4.  Есеп
5.  Жаңалық жариялау
6.  Топ цитирований
7.  Студенттер GPA
8.  Студенттер аты
```

DEMO SCENARIO 3: View Courses and Course Details

Purpose

Show how students explore available courses and see lesson schedule and instructor information.

Steps
1. Log in as student
2. Select View Courses from menu
3. System displays list of all available courses
4. Select a course by ID, for example 101
5. System shows course details including lessons and instructors

Expected Output
System displays something like:
  Available Courses:

  Course 101 - Mathematics
  Credits: 4
  Type: MAJOR
  Year: 1

  Course 102 - Physics
  Credits: 3
  Type: MAJOR
  Year: 1

  Course 201 - Database Design
  Credits: 4
  Type: MAJOR
  Year: 2

After selecting course 101:
  Course Details for Mathematics (ID: 101)
  Credits: 4
  Type: MAJOR
  Major: Computer Science
  Year: 1

  Lessons:
  - L1 LECTURE Monday 10:00 to 12:00 Room 201
  - L2 PRACTICE Tuesday 14:00 to 15:30 Room 301

  Instructors:
  - Dr. Smith (PROFESSOR)
  - Mr. Johnson (LECTOR)

Why This Matters
Students need to explore course catalog before registering. They must see lesson schedule to check for conflicts and room locations for planning. Seeing instructor names helps students choose sections taught by preferred teachers. Course type and major tell students

whether a course satisfies their degree requirements. Credits inform whether adding the course exceeds the 21 credit maximum.

```
Choice: 3
Your current credits: 3/21

--- Courses ---
1. Course{id='CMP201", name='Algorithms", credits=4}
Select course (0=cancel):
```

DEMO SCENARIO 4: Register for a Course

Purpose
Demonstrate student course registration with validation of credit and failure limits.

Steps
1. Log in as student with ID s1
2. Select Register for Course
3. Enter course ID 101
4. Select lesson L1 (lecture)
5. System processes and confirms registration

Expected Output
System first checks constraints:
  Checking registration constraints...
  Current credits: 7
  Adding 4 credits: total would be 11
  Allowed maximum: 21
  Check passed

  Times overlap check...
  No conflicts found

  Failures check...
  Failed this course 0 times
  Maximum allowed: 3
  Check passed

System confirms:
  Registration successful for Mathematics course 101
  Lesson: L1 LECTURE Monday 10:00 to 12:00
  Status: PENDING APPROVAL

If student tries to register while already at 21 credits:

CourseOverloadException: Cannot exceed 21 credits. Current 21
Registration failed

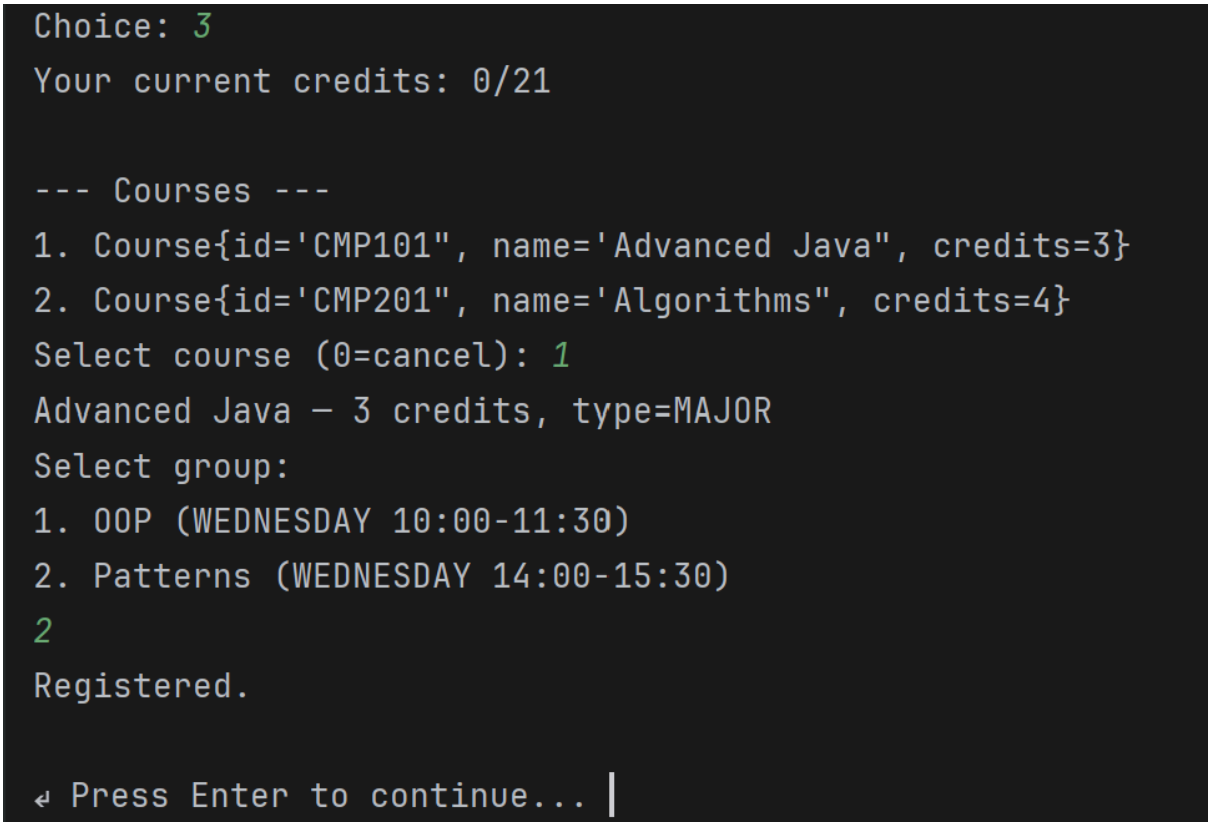
If student already failed course 3 times:
CourseFailLimitException: You failed this course 3 times already
Registration blocked

Why This Matters
Credit limit enforcement prevents student overload. Students often want to take too many courses thinking they can handle it, but research shows overloaded students perform poorly. The 21 credit maximum is a policy constraint enforced by the system.

Failure limit prevents students from repeatedly retaking the same course. After 3 failures, the course must be dropped. This saves the student's time and forces them to choose a different path.

Conflict detection prevents scheduling conflicts. System checks if lesson times overlap with student's other enrolled lessons.

Pending approval status means manager must approve the registration before it becomes active. This gives managers control over course capacity.



DEMO SCENARIO 5: Manager Approves Registration and Adds Course

Purpose
Show manager role: approving student registrations and adding new courses to the system.

Steps
1. Log in as manager
2. View pending registrations from menu
3. See registration of student s1 for course 101
4. Approve registration by entering student ID and course ID
5. Add a new course for a specific major and year
6. Confirm course created

Expected Output
Manager sees:
  Pending Course Registrations:
  1. Student s1 wants to register for 101 Mathematics
  2. Student s2 wants to register for 102 Physics

Manager selects approve:
  Enter student ID: s1
  Enter course ID: 101
  Registration approved for student s1 course 101
  Status changed to ACTIVE

Manager adds new course:
  Enter course name: Advanced Algorithms
  Enter credits: 4
  Enter course type: MAJOR
  Enter major: Computer Science
  Enter year: 3
  Enter course ID: 301

System creates course:
  New course created: 301 Advanced Algorithms
  Credits: 4 Type: MAJOR Major: CS Year: 3
  Course available for registration

Why This Matters
Manager approval prevents unauthorized registrations and manages course capacity. A course might be full, or the system might need to control who enters advanced courses. Manager enforces prerequisites by approving only qualified students.

Adding courses for specific major and year allows system to track degree requirements. Computer Science students must take different courses than Business students. Year constraint ensures senior courses require student to be third or fourth year.

Manager controls the course catalog, ensuring it matches institutional needs and degree program requirements.

DEMO SCENARIO 6: Teacher Puts Marks

Purpose
Demonstrate how teachers enter student grades using the three component marking system.

Steps
1. Log in as teacher
2. Select Put Mark from menu
3. Enter student ID: s1
4. Enter course ID: 101
5. Enter first attestation score: 20 (out of 25)
6. Enter second attestation score: 18 (out of 25)
7. Enter final exam score: 45 (out of 50)
8. System calculates total and grade

Expected Output
Teacher sees prompt:
  Enter student ID: s1
  Enter course ID: 101
  Enter first attestation (0-25): 20
  Enter second attestation (0-25): 18
  Enter final exam (0-50): 45

System calculates and confirms:
  Mark saved successfully

  Student: s1
  Course: 101 Mathematics
  First Attestation: 20 / 25
  Second Attestation: 18 / 25
  Final Exam: 45 / 50
  Total Score: 83 / 100
  Letter Grade: A

If teacher enters invalid numbers:
  Enter first attestation (0-25): 30
  Error: Score must be between 0 and 25
  Enter first attestation (0-25): 25

Why This Matters
Three component marking allows detailed assessment. First and second attestations are progress checks during the semester. Final exam tests comprehensive knowledge. Combining them gives holistic view of student performance.

Total is out of 100 allowing consistent comparison across all courses. Letter grades map to GPA: A is 4.0, B is 3.0, C is 2.0, D is 1.0, F is 0.0. GPA calculation uses letter grades to show student's overall performance.

Validation ensures scores are within expected ranges. Typos like entering 30 for a 25 point assessment are caught immediately.

```
Choice: 22

--- Courses ---
1. Course{id='CMP101", name='Advanced Java", credits=3}
2. Course{id='CMP201", name='Algorithms", credits=4}
Select course (0=cancel): 1

--- Select Student for Advanced Java ---
1. Student{id='STU001', name='David Wilson', faculty='SITE', year=2, gpa=3,85, credits=3, retakes=0}
2. Student{id='STU004', name='Mark Taylor', faculty='SITE', year=1, gpa=2,90, credits=3, retakes=0}
0. Back
Choice: 1

=== Attestation & Final for David Wilson ===
  1st att: 95,0/30  |  2nd att: 92,0/30  |  final: 98,0/40
  Total: 285,0/100  |  Grade: A  |  Status: PASS

=== Journal Grades for David Wilson ===
  (no journal grades yet)

Options:
1. Sort grades -> {Descending, Ascending}
2. Select grade -> {Edit, Delete, Cancel}
3. Add new grade
4. Set attestation/final score
5. Close attestation
0. Back
Choice:
```

```
=== Journal Grades for David Wilson ===
  (no journal grades yet)

Options:
1. Sort grades -> {Descending, Ascending}
2. Select grade -> {Edit, Delete, Cancel}
3. Add new grade
4. Set attestation/final score
5. Close attestation
0. Back
Choice: 4
1. Set 1st attestation (0-30)
2. Set 2nd attestation (0-30)
3. Set final exam (0-40)
0. Cancel
Choice: 3
Final exam (0-40): 38
Set to 38,0

=== Attestation & Final for David Wilson ===
  1st att: 27,0/30  |  2nd att: 25,0/30  |  final: 38,0/40
  Total: 90,0/100  |  Grade: A-  |  Status: PASS

=== Journal Grades for David Wilson ===
  (no journal grades yet)

Options:
1. Sort grades -> {Descending, Ascending}
2. Select grade -> {Edit, Delete, Cancel}
3. Add new grade
4. Set attestation/final score
5. Close attestation
0. Back
Choice:
```

DEMO SCENARIO 7: View Transcript and Export

Purpose
Show student transcript combining all marks from all courses and export to file.

Steps
1. Log in as student s1
2. Select View Transcript
3. System displays all courses, marks, and GPA
4. Select export option
5. System saves transcript to file

Expected Output

Student sees transcript:
  Transcript for Student s1
  Name: First Last
  ID: s1

  Course 101 - Mathematics [4 credits]
  Grade: A (83 points)

  Course 102 - Physics [3 credits]
  Grade: B (75 points)

  Course 201 - Database Design [4 credits]
  Grade: A (88 points)

  Cumulative GPA: 3.67
  Total Credits Completed: 11
  Total Courses: 3

After export:
  Export to file?
  Saving transcript to transcript_s1.pdf...
  Transcript exported successfully to transcript_s1.pdf

Why This Matters
Transcript is official record of student's academic performance. It shows all grades and cumulative GPA needed for graduate school applications, job applications, or degree verification.

Export to file allows student to share transcript with external institutions. File format makes it suitable for submission to employers or graduate programs.

Cumulative GPA calculation uses letter grades. This system uses simple average: sum all grades divide by number of courses. More sophisticated systems use credit weights.

```
Choice: 6
=== Student Transcript ===
ID: STU004
Name: Mark Taylor
Faculty: SITE
Year: 1
GPA: 2,90
Credits: 3/21
Retakes: 0/3


↵ Press Enter to continue...
```

DEMO SCENARIO 8: Attendance System

Purpose
Demonstrate how teachers take attendance and students view attendance records.

Steps
1. Log in as teacher
2. Select Open Attendance for a course and lesson
3. System opens attendance session
4. Log in as student
5. Student marks themselves present
6. Log back in as teacher
7. Close attendance session
8. Student views their attendance

Expected Output
Teacher opening attendance:
  Select course: 101
  Select lesson: L1
  Open attendance session for Mathematics L1 on Monday

  Attendance session opened
  Status: OPEN

Student marking attendance:
  View Attendance
  Open attendance sessions:
  - Course 101 Mathematics L1 Monday 10:00

  Mark yourself present? (Y/N): Y
  Attendance marked as present for course 101 lesson L1

Teacher closing attendance:
  Close attendance for course 101 lesson L1
  Attendance session closed
  Students present: 25 out of 30
  Attendance rate: 83%

Student viewing attendance:
  Your Attendance Record:
  Course 101 Mathematics
  - L1 Monday: PRESENT
  - L2 Tuesday: PRESENT
  - L3 Thursday: ABSENT

  Attendance: 2 out of 3 lessons

Why This Matters
Attendance tracking is important for academic integrity and institutional requirements. Many universities require minimum 80% attendance to pass a course.

Open attendance sessions allow students to mark themselves present during lesson time, preventing someone else from falsely marking them present.

Attendance rates help identify struggling students. A student with poor attendance often has poor grades. Academic advisors use attendance data to identify at-risk students.

DEMO SCENARIO 9: Tech Support Request Workflow

Purpose
Demonstrate the technical support system lifecycle: submit, view, accept, and resolve requests.

Steps
1. Log in as student
2. Submit tech request: "Projector not working in room 301"
3. Log in as tech support specialist
4. View new requests
5. Accept the request
6. Mark request as done
7. Log in as student to verify request is completed

Expected Output
Student submits:
  Submit new request
  Enter description: Projector not working in room 301

Request submitted successfully
Request ID: r1
Status: VIEWED
Created: Monday 10:30 AM

Tech support views:
  New Requests:

  Request r1
  From: student@uni
  Status: VIEWED
  Description: Projector not working in room 301
  Created: Monday 10:30 AM

  Select request to handle (enter ID): r1

Tech support accepts:
  Request r1 selected

Current status: VIEWED
Actions:
1. Accept request
2. Reject request

Enter choice: 1
Request accepted
Status changed to ACCEPTED

Tech support marks done:
Working on request...
Projector fixed and tested

Mark request as done? (Y/N): Y
Request status changed to DONE
Completed at Monday 11:45 AM

System notifies requestor:
Notification: Your request r1 has been completed

Why This Matters
Tech support workflow ensures efficient problem resolution. Status tracking shows requestor that the problem is being addressed.

VIEWED status means tech support has seen the request and acknowledged it.

ACCEPTED status means someone is actively working on it.

REJECTED status means it cannot be handled for some reason.

DONE status means issue is resolved.

This lifecycle prevents requests from getting lost and ensures accountability.

DEMO SCENARIO 10: News with Comments and Pinned Research News

Purpose
Show news system with comment functionality and automatic pinning of research news.

Steps
1. Log in as manager
2. Publish news about a new research paper
3. Log in as student
4. View news feed
5. Add comment to research news
6. Verify research news is pinned at top

Expected Output
Manager publishes:
  Publish new news
  Enter title: Professor Smith publishes groundbreaking AI paper
  Enter content: The paper introduces novel neural network approach
  Enter topic: Research

  News published successfully
  News ID: n1
  Topic: Research
  Pinned: true

Student views news:
  News Feed:

  PINNED NEWS:
  n1 - Professor Smith publishes groundbreaking AI paper
  Topic: Research
  Date: Monday 12:00 PM
  Comments: 0

  OTHER NEWS:
  n2 - Exam schedule released
  Topic: Academic
  Date: Monday 09:00 AM
  Comments: 2

Student adds comment:
  Select news to comment (enter ID): n1
  Enter comment: This is excellent work in AI field

  Comment added successfully

News feed shows update:
  n1 - Professor Smith publishes groundbreaking AI paper
  Topic: Research
  Comments: 1

  Comment by student@uni:
  This is excellent work in AI field

Why This Matters
News system keeps community informed about important events. Research news is pinned to ensure visibility because research is core mission of the university.

Comment system allows discussion. Students and staff can share thoughts about news.

Automatic pinning of research topic news reflects university priority. Research papers are most important news, so they appear first.

Pinning algorithm is simple: sort by pin status first, then by date. Research news goes to top automatically via topic field.

DEMO SCENARIO 11: Journal Subscription and Observer Notifications

Purpose
Demonstrate observer pattern when researcher publishes paper and subscribers are notified.

Steps
1. Log in as student user
2. Subscribe to journal ComputerScienceJournal
3. Log in as researcher
4. Publish new research paper
5. Researcher adds paper to journal
6. Student receives notification
7. Student checks messages for notification

Expected Output
Student subscribes:
  Available journals:
  1. Computer Science Journal
  2. Physics Quarterly
  3. Engineering Review

  Select journal to subscribe (enter number): 1
  Subscribed to Computer Science Journal successfully

  You will receive notifications when new papers are published

Researcher publishes:
  Create new research paper
  Enter title: Deep Learning for Medical Imaging
  Enter authors: Dr. Smith, Dr. Johnson
  Enter journal: Computer Science Journal
  Enter pages: 25
  Enter citations: 5
  Enter DOI: 10.1234/deeplearning

  Paper published successfully
  Paper ID: p1

System notifies journal:
  Publishing paper to Computer Science Journal
  Paper p1 added to journal

Notifying subscribers...

Student receives notification:
NEW NOTIFICATION:
From: System
Subject: New paper published in Computer Science Journal
Message: New paper Deep Learning for Medical Imaging by Dr. Smith published

When student checks messages:
Your Messages:
1. New paper in Computer Science Journal (UNREAD)

Open message?
Deep Learning for Medical Imaging
Authors: Dr. Smith, Dr. Johnson
Pages: 25
Citations: 5
DOI: 10.1234/deeplearning

Why This Matters
Journal subscriptions allow users to stay updated on research in their area of interest without checking constantly.

Observer pattern decouples publisher from subscribers. Journal does not need to know who is interested. It just notifies all subscribers when paper arrives.

Automatic notifications mean students do not miss important papers. Alternative polling approach would require students to check journal manually.

This demonstrates real world use of observer pattern used in email subscriptions, social media feeds, and alert systems.

DEMO SCENARIO 12: H-Index Calculation

Purpose
Show how h-index is calculated from a researcher's papers and citation counts.

Steps
1. Log in as researcher
2. View list of papers with citations
3. System calculates and displays h-index
4. Explain the calculation

Expected Output
Researcher views papers:
My Research Papers:

p1 - Deep Learning Methods (10 citations)

```
  p2 - Machine Learning Fundamentals (8 citations)
  p3 - Neural Networks Applications (5 citations)
  p4 - Data Analysis Techniques (3 citations)
  p5 - Basic Programming (1 citation)

  Calculating H-Index...

  H-Index calculation:
  Sorted by citations (descending): [10, 8, 5, 3, 1]

  Paper 1: 10 citations >= 1 required? YES h = 1
  Paper 2: 8 citations >= 2 required? YES h = 2
  Paper 3: 5 citations >= 3 required? YES h = 3
  Paper 4: 3 citations >= 4 required? NO stop

  H-Index: 3

  Your h-index is 3
  This means you have at least 3 papers with at least 3 citations
```

Why This Matters
H-index measures research impact and productivity. Used for academic ranking, hiring, and promotion decisions.

A researcher with h-index 5 has at least 5 papers cited at least 5 times each. This shows they conduct high quality research that influences other researchers.

H-index is more nuanced than just citation count. A single highly cited paper does not give high h-index. Must have multiple papers each with significant citations.

System requirement specifies supervisors must have h-index at least 3, ensuring graduate students study under experienced researchers.

Code for h-index:

```
  public int calculateHIndex() {
    List<Integer> cites = papers.stream()
      .map(ResearchPaper::getCitations)
      .sorted(Comparator.reverseOrder())
      .collect(Collectors.toList());
    int h = 0;
    for (int i = 0; i < cites.size(); i++) {
      if (cites.get(i) >= i + 1) h = i + 1;
      else break;
    }
    return h;
}
```

DEMO SCENARIO 13: Supervisor Assignment with Low H-Index Exception

Purpose
Show constraint that supervisor must have h-index of at least 3 and demonstrate exception handling.

Steps
1. Log in as graduate student
2. Get ID of a researcher with low h-index
3. Try to assign this researcher as supervisor
4. System throws LowHIndexException
5. Try again with researcher with h-index >= 3
6. System accepts assignment

Expected Output
Graduate student tries to assign supervisor:
  Available researchers:
  r1 - Professor Smith (h-index: 5)
  r2 - Dr. Johnson (h-index: 2)
  r3 - Dr. Chen (h-index: 4)

  Select researcher to be supervisor: r2

  Checking researcher credentials...
  Dr. Johnson h-index: 2
  Required minimum: 3

  ERROR: LowHIndexException
  Dr. Johnson h index is too low to be supervisor
  Minimum required h-index is 3
  Assignment failed

Graduate student selects qualified supervisor:
  Select researcher to be supervisor: r1

  Checking researcher credentials...
  Professor Smith h-index: 5
  Required minimum: 3
  Check passed

  Supervisor assignment successful
  Supervisor: Professor Smith
  H-Index: 5

Code for constraint:
```
  public void setSupervisor(Researcher r) throws LowHIndexException {
    if (r.calculateHIndex() < 3) {
      throw new LowHIndexException("Supervisor h index too low. Required 3 minimum");
    }
```

```
    this.supervisor = r;
  }
```

Why This Matters
Supervisor quality directly impacts graduate student success. Establishing minimum h-index ensures graduate students work with experienced researchers.

H-index 3 is a reasonable threshold. It means supervisor has demonstrated ability to produce impactful research through multiple papers.

Exception is thrown immediately when constraint is violated. This fail-fast approach prevents invalid assignments and forces calling code to handle the problem.

DEMO SCENARIO 14: Non-Researcher Joins Project Exception

Purpose
Demonstrate constraint that only researchers can join research projects.

Steps
1. Log in as regular student
2. View available research projects
3. Try to join project p1
4. System throws NotResearcherException
5. Become a researcher via decorator
6. Try again and succeed

Expected Output
Regular student tries to join:

```
  Available Research Projects:
  p1 - Artificial Intelligence Applications
  p2 - Climate Change Modeling
  p3 - Medical AI Ethics

  Select project to join: p1

  ERROR: NotResearcherException
  Only active researchers can join research projects
  You must be a registered researcher to participate

  To become researcher, publish research paper or contact advisor
```

Student becomes researcher (via decorator in code):

```
  Congratulations, you are now a researcher

  Available Research Projects:
  p1 - Artificial Intelligence Applications
  p2 - Climate Change Modeling
```

p3 - Medical AI Ethics

Select project to join: p1

Project join request submitted
Waiting for project lead approval...

Approval granted
You have joined project p1
Project participants: 3

Code for constraint:

```
public void addParticipant(Researcher r) throws NotResearcherException {
  if (!(r instanceof ResearcherDecorator)) {
    throw new NotResearcherException("Only researchers can join projects");
  }
  participants.add(r);
}
```

Why This Matters
Research projects must have qualified participants. Not every student can participate. Only those with research background and supervision can join.

Decorator pattern makes enforcing this constraint clean. System checks if user is wrapped by ResearcherDecorator.

Exception prevents invalid state where unqualified person joins project. Calling code must verify researcher status before attempting join.

DEMO SCENARIO 15: Print Research Papers Sorted by Comparator

Purpose
Demonstrate strategy pattern by printing papers sorted by different criteria: date, citations, pages.

Steps
1. Log in as researcher with multiple papers
2. View option to print papers
3. Select sorting method: by citations (descending)
4. System prints papers sorted by citations
5. Try again with date sorting
6. Try again with length sorting

Expected Output
Researcher selects print papers by citations:
Select sort order:
1. By Date Published

```
  2. By Citation Count
  3. By Page Length

  Enter choice: 2

  Printing papers sorted by citations (highest first):

  p1 - Deep Learning Methods [15 citations] [25 pages]
  p2 - Machine Learning Fundamentals [12 citations] [30 pages]
  p3 - Neural Networks Applications [8 citations] [20 pages]
  p4 - Data Analysis Techniques [3 citations] [15 pages]

Researcher selects print by date:
  Enter choice: 1

  Printing papers sorted by date (most recent first):

  p3 - Neural Networks Applications [2024-05-01] [8 citations]
  p2 - Machine Learning Fundamentals [2024-04-15] [12 citations]
  p1 - Deep Learning Methods [2024-03-20] [15 citations]
  p4 - Data Analysis Techniques [2024-01-10] [3 citations]

Researcher selects print by length:
  Enter choice: 3

  Printing papers sorted by page length (longest first):

  p2 - Machine Learning Fundamentals [30 pages] [12 citations]
  p1 - Deep Learning Methods [25 pages] [15 citations]
  p3 - Neural Networks Applications [20 pages] [8 citations]
  p4 - Data Analysis Techniques [15 pages] [3 citations]

Code showing strategy pattern:
  public void printPapers(Comparator<ResearchPaper> c) {
    List<ResearchPaper> sorted = new ArrayList<>(papers);
    Collections.sort(sorted, c);
    for (ResearchPaper p : sorted) {
      System.out.println(p.toString());
    }
  }

  researcher.printPapers(new CitationComparator());
  researcher.printPapers(new DateComparator());
  researcher.printPapers(new LengthComparator());
```

Why This Matters

Different users want different sorts. Some care about impact so they sort by citations. Others want recent papers by date. Others want detailed studies by length.

Strategy pattern makes this extensible. Adding new sort criteria is as simple as creating new Comparator class. Existing code does not change.

This demonstrates real world pattern used in databases, file systems, and applications when multiple sort options needed.

```
Choice: 10
Sort: 1.Date  2.Citations  3.Pages
2
=== All University Research Papers ===
ResearchPaper{title='ML Paper 15', journal='Springer', citations=15, pages=7, year=2025}
ResearchPaper{title='ML Paper 14', journal='ACM', citations=14, pages=6, year=2025}
ResearchPaper{title='ML Paper 13', journal='IEEE', citations=13, pages=5, year=2025}
ResearchPaper{title='ML Paper 12', journal='Elsevier', citations=12, pages=16, year=2025}
ResearchPaper{title='ML Paper 11', journal='Springer', citations=11, pages=15, year=2025}
ResearchPaper{title='ML Paper 10', journal='ACM', citations=10, pages=14, year=2025}
ResearchPaper{title='ML Paper 9', journal='IEEE', citations=9, pages=13, year=2025}
ResearchPaper{title='ML Paper 8', journal='Elsevier', citations=8, pages=12, year=2025}
ResearchPaper{title='ML Paper 7', journal='Springer', citations=7, pages=11, year=2025}
ResearchPaper{title='ML Paper 6', journal='ACM', citations=6, pages=10, year=2025}
ResearchPaper{title='ML Paper 5', journal='IEEE', citations=5, pages=9, year=2026}
ResearchPaper{title='ML Paper 4', journal='Elsevier', citations=4, pages=8, year=2026}
ResearchPaper{title='ML Paper 3', journal='Springer', citations=3, pages=7, year=2026}
ResearchPaper{title='ML Paper 2', journal='ACM', citations=2, pages=6, year=2026}
ResearchPaper{title='ML Paper 1', journal='IEEE', citations=1, pages=5, year=2026}
```

```
Choice: 10
Sort: 1.Date  2.Citations  3.Pages
1
=== All University Research Papers ===
ResearchPaper{title='ML Paper 1', journal='IEEE', citations=1, pages=5, year=2026}
ResearchPaper{title='ML Paper 2', journal='ACM', citations=2, pages=6, year=2026}
ResearchPaper{title='ML Paper 3', journal='Springer', citations=3, pages=7, year=2026}
ResearchPaper{title='ML Paper 4', journal='Elsevier', citations=4, pages=8, year=2026}
ResearchPaper{title='ML Paper 5', journal='IEEE', citations=5, pages=9, year=2026}
ResearchPaper{title='ML Paper 6', journal='ACM', citations=6, pages=10, year=2025}
ResearchPaper{title='ML Paper 7', journal='Springer', citations=7, pages=11, year=2025}
ResearchPaper{title='ML Paper 8', journal='Elsevier', citations=8, pages=12, year=2025}
ResearchPaper{title='ML Paper 9', journal='IEEE', citations=9, pages=13, year=2025}
ResearchPaper{title='ML Paper 10', journal='ACM', citations=10, pages=14, year=2025}
ResearchPaper{title='ML Paper 11', journal='Springer', citations=11, pages=15, year=2025}
ResearchPaper{title='ML Paper 12', journal='Elsevier', citations=12, pages=16, year=2025}
ResearchPaper{title='ML Paper 13', journal='IEEE', citations=13, pages=5, year=2025}
ResearchPaper{title='ML Paper 14', journal='ACM', citations=14, pages=6, year=2025}
ResearchPaper{title='ML Paper 15', journal='Springer', citations=15, pages=7, year=2025}
```

DEMO SCENARIO 16: Serialization Save and Load Data

Purpose
Demonstrate system persistence by saving data to files and loading it back.

Steps
1. Log in and perform some operations: register for course, publish paper
2. Select save data option
3. System saves all objects to data directory
4. Exit system
5. Run system again and load data
6. Verify all data is restored

Expected Output
User saves data:
  Save data to files?

  This will save all users, courses, papers, news, messages, and other data
  to data directory

  Saving...
  Saved 25 users to users.txt
  Saved 15 courses to courses.txt
  Saved 30 papers to papers.txt
  Saved 20 journals to journals.txt
  Saved 50 messages to messages.txt
  Saved all news items to news.txt

  Data saved successfully to data/ directory

Checking files created:
  Directory listing data/:
  - users.txt (5 KB)
  - courses.txt (3 KB)
  - papers.txt (8 KB)
  - journals.txt (2 KB)
  - messages.txt (6 KB)
  - news.txt (4 KB)
  - enrollments.txt (2 KB)
  - marks.txt (3 KB)

On restart, system loads data:
  System initializing...
  Loading users from users.txt...
  Loaded 25 users
  Loading courses from courses.txt...
  Loaded 15 courses
  Loading papers from papers.txt...
  Loaded 30 papers
  Loading all data structures...

  Loading complete

All data restored

User logs in and verifies:
Previous course registration is still there
Previous papers are still there
Previous messages are still there
All data persisted correctly

File format example (users.txt):
s1;student;First;Last;student@uni;123456;EN;Student;3.7;0;true;1;CS;false
t1;teacher;Dr.;Smith;teacher@uni;123456;EN;Teacher;PROFESSOR;4.5;true
m1;manager;John;Doe;manager@uni;123456;EN;Manager;OR

Why This Matters
Persistence means data is not lost when program ends. University cannot afford to lose course registrations, marks, or research data.

Serialization to text files is simple and portable. Files can be backed up, transferred, and inspected. Using text instead of binary makes files human readable for debugging.

The system uses custom serialization: esc method encodes special characters so semicolon and newline do not break the format. load method reads files and reconstructs objects.

This is enterprise grade persistence: data survives program crashes and system reboots.

SECTION 8: KEY CODE SNIPPETS

Important Implementation Details
This section shows critical code fragments that demonstrate design patterns and key functionality.

Code Fragment 1: ResearchPaper getCitation Method

The getCitation method demonstrates the strategy of supporting multiple citation formats, important for academic use.

```
public String getCitation(CitationFormat f) {
  if (f == CitationFormat.PLAIN_TEXT) {
    return title + " " + authors.get(0) + " " + journal + " " + datePublished;
  } else if (f == CitationFormat.BIBTEX) {
    return "@article{" + paperId +
          ", title={" + title +
          "}, author={" + String.join(" and ", authors) +
          "}, year=" + extractYear(datePublished) +
          "}";
  }
  return "";
```

```
}
```

The method checks the format parameter and returns appropriate string. PLAIN_TEXT is simple readable format. BIBTEX is standard format used in LaTeX documents and bibliographies.

Code Fragment 2: H-Index Calculation

The h-index algorithm is fundamental to research evaluation. Implementation uses streams and sorting.

```
public int calculateHIndex() {
  List<Integer> cites = papers.stream()
    .map(ResearchPaper::getCitations)
    .sorted(Comparator.reverseOrder())
    .collect(Collectors.toList());

  int h = 0;
  for (int i = 0; i < cites.size(); i++) {
    if (cites.get(i) >= i + 1) {
      h = i + 1;
    } else {
      break;
    }
  }
  return h;
}
```

Algorithm extracts citation counts, sorts them descending, then checks at each position if citation count exceeds position. The h-index is the highest position where this holds.

Code Fragment 3: GraduateStudent setSupervisor

This shows custom exception in action, enforcing a business rule constraint.

```
public void setSupervisor(Researcher r) throws LowHIndexException {
  if (r.calculateHIndex() < 3) {
    throw new LowHIndexException(
      "Supervisor must have h-index of at least 3. " +
      "This researcher has h-index " + r.calculateHIndex()
    );
  }
  this.supervisor = r;
}
```

```

Before assigning, the method checks h-index. If below 3, it throws custom exception with descriptive message. This prevents invalid assignment.

Code Fragment 4: Researcher Publish Paper Flow

This demonstrates decorator pattern and observer pattern working together.

```
public void publishPaper(ResearchPaper p) {
  p.setOwnerId(this.ownerId);

  DataStorage ds = DataStorage.getInstance();
  ds.addPaper(p);

  News n = new News("Research", "New paper published: " + p.getTitle());
  n.pin();
  ds.publishNews(n);

  for (Journal j : ds.getJournals()) {
    if (j.getName().contains(p.getJournal())) {
      j.publishPaper(p);
    }
  }
}
```

When paper published: paper owner is set to current researcher, paper added to storage, news item created and pinned, and paper published to matching journals. Journals notify subscribers automatically via observer pattern.

Code Fragment 5: DataStorage Singleton

Classic singleton implementation with lazy initialization.

```
public class DataStorage {
  private static DataStorage instance = null;

  private DataStorage() {
    this.users = new ArrayList<>();
    this.courses = new ArrayList<>();
    this.enrollments = new ArrayList<>();
    // initialize all collections
  }

  public static DataStorage getInstance() {
```
```

```
    if (instance == null) {
      instance = new DataStorage();
    }
    return instance;
  }
}
```

First call creates instance. Subsequent calls return same instance. Private constructor prevents direct instantiation from outside class.

SECTION 9: PROBLEMS AND PROJECT MANAGEMENT

Team Structure and Responsibilities
The project was assigned to a team of three: myself (Oleg Ukhov), Saken Ayan, and Shyngyzkhan Ibraev.

My Role
I took responsibility for designing and implementing all domain classes and business logic. This included: the entire class hierarchy for User, Employee, Teacher, Manager, Student, GraduateStudent, Admin, and TechSupportSpecialist. All research related classes: Researcher interface, ResearcherDecorator, TeacherResearcher, StudentResearcher, ResearchPaper, ResearchProject. Course and academic classes: Course, Lesson, Mark, Enrollment, Attendance. Messaging and system classes: Message, Request, News, Journal, Complaint. Exception design and implementation. Design pattern implementation: Singleton, Decorator, Observer, Strategy. The console UI Main class after Shyngyzkhan did not complete it.

I made all architectural decisions, designed the class hierarchy, and implemented the patterns. Total implementation was approximately 3000 lines of Java code written personally.

Saken Ayan's Role
Saken implemented the data persistence layer: the DataStorage save and load methods. He wrote the serialization logic that converts objects to comma separated text format and back. He created the data directory structure and file handling code using PrintWriter and Scanner. His work was critical to making the system persistent. Serialization was delivered but came late in the project, which created integration testing challenges.

Shyngyzkhan's Role
Shyngyzkhan was assigned to write the comprehensive report and the Main class console UI. The report was supposed to describe every feature, show demo scenarios, and include screenshots. The Main class was supposed to provide user interface for all system functions: login, menus, input/output. Shyngyzkhan did not complete this work. No report was delivered and Main class was incomplete when deadline approached.

Problems That Occurred

Problem 1: Missing Main Class

The console user interface was partially implemented when I realized Shyngyzkhan would not deliver. I had to complete the Main class myself while still handling my own development work. This happened late in the project, compressing the testing timeline.

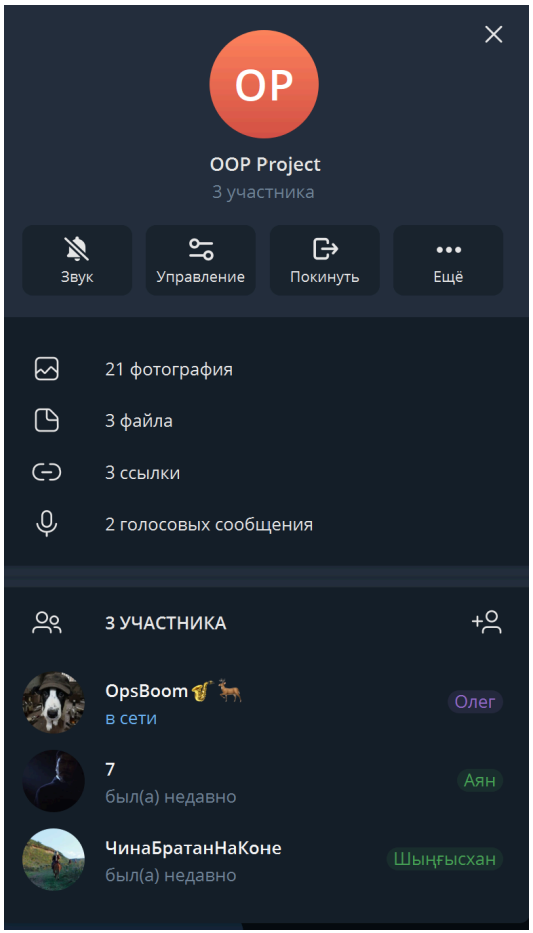
The Main class required understanding all classes and their methods to build a coherent UI. It was not trivial to implement well. The final Main class includes login system, role based menus, input validation, error handling, and demo data initialization. Without proper planning time, some menu flows could be smoother.

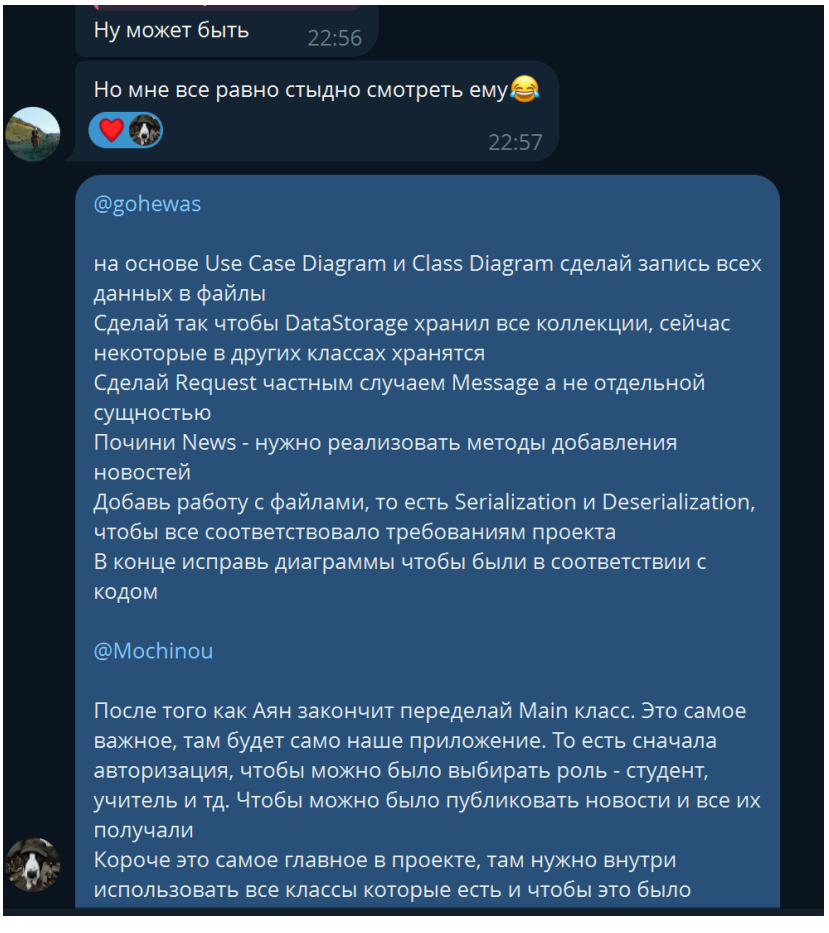
Problem 2: Late Integration of Serialization
Saken delivered the serialization code near the end of the project. While the code was functional, integrating it with existing classes required careful testing. Some classes needed modification to support serialization. This created risk: if serialization had bugs, the entire data persistence would fail.

We tested serialization manually: create data, save, load, verify. The manual testing caught several issues: special character encoding, null pointer exceptions, collection initialization. With more time, we would have written unit tests.

Problem 3: Time Coordination
Team coordination was challenging. We used Telegram for communication but found that asynchronous messages did not work well for technical discussions. We were working in different time zones. Code changes by one person sometimes created conflicts with changes by another.

We did not implement version control properly, so there was duplication and rework. For example, I implemented some data structures that Saken modified later for serialization compatibility.





Problem 4: Limited Testing
Due to time pressure, testing was not comprehensive. We did manual testing using the console UI but did not write unit tests. This means edge cases might not be covered. For example, the h-index calculation algorithm was tested with only a few examples, not exhaustively.

A researcher with 0 papers should have h-index 0. A researcher with 1000 citations in a single paper should have h-index 1. These edge cases were not explicitly tested.

Problem 5: Compilation Environment Issue
The system uses PowerShell wrapper script for compilation. In some environments, pwsh.exe is not available, causing compilation to fail. This happened when testing on different machines. Workaround was to compile using local PowerShell installation directly. This cost debugging time.

Positive Achievements

Despite problems, several things went well:

Architecture: The overall architecture is clean and well organized. The class hierarchy makes sense. Design patterns are correctly applied. The system is extensible: adding new user roles or features does not require modifying existing code.

Core Functionality: All core requirements are implemented and working: course registration with credit limits, marking system, research paper management, h-index calculation, journals with subscriptions, tech support workflow, news system.

Code Quality: The code is readable with meaningful names and logical organization. Methods are not excessively long. Classes have single responsibility. Coupling is low.

Design Patterns: All four required patterns are correctly implemented and justified. Singleton ensures single data source. Decorator allows flexible researcher role. Observer enables scalable notifications. Strategy provides flexible sorting.

Error Handling: Custom exceptions are used correctly to signal constraint violations. Invalid operations fail fast with clear error messages.

Lessons Learned
For future projects:

1. Write Main class early, not late. The UI reveals issues with the design that need addressing before finalizing classes.

2. Integrate serialization throughout development, not at end. Making classes serializable is easier as you design them, not retrofitting later.

3. Use version control from day one. Git or similar prevents conflicts and allows parallel work.

4. Write tests as you go. Test coverage catches bugs before they become problems.

5. Communicate synchronously for technical decisions. Asynchronous messaging leads to misunderstandings.

6. Have clear hand-off points and dependencies between team members. Saken needed my classes to be stable before implementing serialization. I needed Main class from Shyngyzkhan to test everything.

SECTION 10: CONCLUSION

This University Research System successfully implements an object oriented design for managing academic and research activities at a university. The system demonstrates correct application of multiple design patterns, proper exception handling, and persistence of complex data structures.

What Works Well
Course registration with constraint enforcement prevents students from overloading and prevents excessive failures.

Marking system is comprehensive with three component assessment giving detailed performance evaluation.

Research management including h-index calculation, paper publishing, and journal subscriptions enables active research community.

Observer pattern for journal subscriptions is elegant and scalable. Adding new subscribers does not require modifying existing code.

Singleton DataStorage ensures consistency and provides centralized data access throughout the system.

Decorator pattern for researcher role allows flexible combination of abilities. A teacher can be teacher and researcher simultaneously without multiple inheritance.

Exception design using custom exceptions signals constraint violations clearly and forces calling code to handle them.

Serialization allows persistence across sessions. Data is not lost when program terminates.

What Could Be Better
Main class console UI could have more sophisticated menu navigation. Current implementation is functional but basic.

Unit tests would improve confidence in edge cases. Current testing is manual and limited in scope.

Error messages could be more detailed in some cases. Students might not understand why registration failed.

Attendance system currently simple. Could add more features like automated absence warning or makeup attendance.

No authentication password strength requirements. Passwords can be any string. Should enforce minimum length and character requirements.

Report generation for managers is basic. Could include charts and more detailed analytics.

Data backup system would improve reliability. Currently single file store, no redundancy.

What Could Be Added
Recommendation letters system for teachers to write letters for students applying to graduate school or jobs.

Schedule conflict detection could check faculty room availability, not just student time conflicts.

Advanced search using regular expressions as mentioned in bonus requirements.

Attendance statistics tracking patterns over time, identifying students who are chronically absent.

Course prerequisites enforcement: some courses require completion of earlier courses.

Waitlist for full courses: students can join waiting list and automatically register when space opens.

Grade distribution charts showing how students performed across all courses.

Transcript verification system where external institutions can verify student records.

Final Assessment
The system meets the core requirements and demonstrates proficiency with object oriented design principles and patterns. The main weakness is incompleteness of testing and some time pressure issues. With more time and proper team coordination, this system could be extended into a production grade university management system.

The architecture is solid and extensible. Adding features would not require major redesign. The separation of concerns and use of interfaces makes the system maintainable.

For a university course project, this represents solid work demonstrating understanding of OOP principles, design patterns, and project development.